

Implementation Plan: Phase 0 – Project Initialization & Repository Setup

System specifications and architectural roadmap for Phase 0 Initialization of the PERN/PostgreSQL Helpdesk System with Knowledge Base Management, AI Service Architecture, and AI Governance.

1. Project Vision

To build a SaaS-ready, enterprise-grade AI Helpdesk System that integrates traditional support ticketing with advanced natural language processing. The system will automate administrative flows, augment agent workloads with smart suggestions, deflect support volume using a structured Knowledge Base, and analyze communication sentiments to optimize service quality and prioritization.

2. Project Objectives

- **Volume Deflection:** Suggest highly relevant solution articles and resolved cases before ticket submission.
- **Intelligent Routing:** Automate classification of incoming tickets into categories and priorities.
- **Sentiment-Driven Dispatch:** Analyze customer sentiments to flag frustrated users and recommend escalations.
- **Agent Assistance:** Provide automated timeline summaries and draft contextual replies.
- **Rigorous Governance:** Enforce confidence score metrics, manual overrides, decision logging, and responsible AI guardrails.
- **Database Coherency:** Utilize a normalized relational model (PostgreSQL) with type-safe access layers (Prisma).

3. Project Scope

The scope covers PostgreSQL and Prisma database planning, versioned Node.js/Express.js backend development, Knowledge Base Management modules, React.js frontend design,

integration with Gemini AI via a dedicated AI Service Layer, local/cloud storage management, and deployment/logging orchestration.

4. Functional Scope

- **Authentication & User Management:** Secure token-based access with role controls (Admin, Agent, Customer).
- **Knowledge Base Management:** Management of articles and FAQs with categories/tags, admin CRUD interfaces, publishing workflows, version histories, and visibility rules.
- **Ticketing Operations:** Creation, priority updates, category mapping, assignments, and life-cycle status transitions.
- **Communication Threading:** Comments and internal agent-only notes.
- **AI Knowledge Base Recommendation:** Pre-creation search against the Knowledge Base and resolved tickets using semantic overlap to deflect submissions.
- **AI Sentiment Analysis:** Real-time evaluation of descriptions and comments to detect emotional states (Positive, Neutral, Negative, Frustrated, Urgent) for priority assistance.
- **AI Core features:** Auto-categorization, priority prediction, duplicate warnings, drafting suggestions, and summarizing timelines.
- **Audit Trails:** Capture details of ticket events, assignments, profile updates, and AI decisions.

5. Non-Functional Scope

- **API Performance:** Target latencies under 50ms for database CRUD queries, under 150ms for vector search, and under 300ms for active AI queries.
- **System Scalability:** Connection pooling and caching architecture (Redis) to support high concurrent usage.
- **Security & Isolation:** Enforce row-level restrictions and parameter validation.
- **AI Fault Tolerance:** Graceful fallback strategies to keyword indexing when external models time out.

6. Technology Stack Selection (with justification)

- **Frontend: React, Tailwind CSS, Axios, React Router:** High performance layouts, styling flexibility, and standard request integrations.

- **Backend: Node.js, Express.js:** Non-blocking execution paths and simple middleware bindings.
- **Database: PostgreSQL:** Strong relational consistency and reliable query engines.
- **ORM: Prisma:** Type-safe database queries and automated migration histories.
- **AI: Google Gemini API & Sentence Embeddings:** Rich NLP analysis and semantic similarity computations.
- **Caching: Redis:** High throughput in-memory caching to optimize database queries and rate limiter states.

7. System Architecture Overview

The application follows a modular, layered architecture:

```
graph TD;
    A[React Client (Presentation Layer)] --> B[Express API (Routing & Validation Layer)];
    B --> C[Business Services (Logical Controller Layer)];
    C --> D[AI Service Layer (Advanced Processing Layer)];
    D --> E[Prisma ORM (Database Abstraction Layer)];
    E --> F[PostgreSQL (Storage Layer)];
```

External Services: Gemini API, Cloudinary (file storage), Email gateway, Winston Logger, and Redis cache.

8. High-Level Architecture Diagram (Conceptual)

Flow: Customer Browser -> React client -> Express server router -> Controllers & Business Services -> AI Service Layer -> Prisma ORM -> PostgreSQL. Services interact with Redis and Winston. AI Layer calls Gemini API.

9. Repository Strategy

Monorepo structure to keep client and server directories in the same project, ensuring simplified coordination of versions and deployments.

10. Git Branching Strategy

GitFlow process where features are built in short-lived feature branches, merged into a develop branch, and finally pushed to the main production branch.

11. Monorepo Structure Planning

Organization dividing source directories into frontend client workspace folders and backend server workspace folders.

12. Folder Organization Strategy

Maintain modular sub-directories. Server: routes, controllers, middleware, config, services, and database schemas. Client: components, pages, routes, styles, and utilities.

13. Naming Conventions

CamelCase for React components. camelCase for functions and variables. snake_case for PostgreSQL database elements. lowercase and hyphens for branch naming.

14. Coding Standards

Strict use of ESLint, Prettier formatter, and async/await error catching pipelines to prevent uncaught exceptions.

15. Documentation Strategy

Maintain detailed API specs, setup files (README), and inline JSDoc summaries explaining parameters and models.

16. Environment Configuration Strategy

Configured using env files, with environment variables checks on startup and templates for setting environment variable values.

17. Configuration Management

Centralized configuration reader script checks environmental variables and prevents boots when key credentials are missing.

18. Security Guidelines

Encryption of tokens, rate limiting IP origins, sanitized inputs, and Helmet policy settings.

19. Database Strategy (PostgreSQL + Prisma)

PostgreSQL schema migrations tracked in version control, type-safe queries compiled via Prisma Client, and relational keys verified at the DB layer.

20. AI Integration Strategy

Gemini SDK calls using isolated prompts, vector embeddings for similarity checking, with rules-based backup scripts to support standard ticket routing if API calls time out.

21. File Storage Strategy

Local storage pathways for development and Cloudinary integrations for staging and production, serving files through authorization validations.

22. Logging Strategy

Structured logs saved as JSON strings with levels (info, warn, error) using automatic log rotation files.

23. Error Handling Strategy

Centralized backend error handlers capture and map exceptions, shielding detailed errors from clients.

24. Deployment Strategy

React client compiled to static assets (Vercel/Netlify). Express backend deployed on hosting servers. PostgreSQL managed by DB providers (Neon/Supabase).

25. CI/CD Planning

Actions checking lint errors, running test scripts, and compiling front/back builds before merging PRs.

26. Third-Party Services

PostgreSQL databases, Google Gemini NLP model, Cloudinary media storage, Redis caching, and email gateways (Resend/SendGrid).

27. Development Workflow

Develop branch checkout -> Local migrations update -> Local checks -> PR creation -> Code review -> Merge to develop.

28. Team Collaboration Guidelines

Enforce mandatory code review processes requiring peer approval, and maintain task boards outlining features status.

29. Versioning Strategy

Utilize Semantic Versioning models and version API endpoints under /api/v1.

30. Testing Strategy

Test unit logic, verify API routes using integration scripts, and simulate user workflows with E2E scripts.

31. AI Service Architecture

To isolate external API processing and allow horizontal scaling, we deploy a dedicated AI Service Layer in the backend application:

- **AI Service:** Orchestrates specialized sub-services.
- **Prompt Manager:** Controls systemic prompts templates and variable injections.
- **Embedding Service:** Requests vector representations.
- **Knowledge Retrieval Service:** Performs similarity comparisons against database articles.
- **Recommendation Service:** Evaluates retrieval distances to suggest articles/resolutions.
- **Sentiment Service:** Parses emotional cues (Positive, Neutral, Negative, Frustrated, Urgent).
- **Summary Service:** Compiles summaries of descriptions and comments.
- **Categorization Service:** Maps tickets to taxonomical categories.
- **Duplicate Detection Service:** Flags active duplicates using embedding thresholds.

32. AI Governance

To verify reliability, safety, and transparency in our automation pipelines:

- **Confidence Scores:** Every AI prediction must output a confidence value between 0.0 and 1.0.
- **Human Review & Override:** Support agents can modify any AI-applied tag, which is tracked.
- **AI Decision Logging:** Write permanent log records for every AI evaluation (tokens, scores, timestamps, overrides).
- **Failure Handling & Fallback:** If Gemini times out, the service defaults to regular regex keyword mapping and flags tickets for review.
- **Responsible AI:** Sanitize inputs to strip PII before API calls.

33. Milestones

- **Phase 0:** Project Initialization & Repository Setup
- **Phase 1:** Project Setup
- **Phase 2:** Authentication & Authorization
- **Phase 3:** User Management
- **Phase 4:** Knowledge Base Management (Admin & Publishing Workflows)
- **Phase 5:** Ticket Management (CRUD)

- **Phase 6:** Categories & Priorities
- **Phase 7:** Ticket Assignment Workflow
- **Phase 8:** Comments & Attachments
- **Phase 9:** Search, Filtering & Sorting
- **Phase 10:** AI Ticket Categorization
- **Phase 11:** AI Priority Prediction
- **Phase 12:** AI Duplicate Ticket Detection
- **Phase 13:** AI Knowledge Base Recommendation
- **Phase 14:** AI Suggested Replies
- **Phase 15:** AI Ticket Summarization
- **Phase 16:** AI Sentiment Analysis
- **Phase 17:** Dashboard & Analytics (incorporating sentiment trends)
- **Phase 18:** Notifications (Email/In-App)
- **Phase 19:** Activity Logs & Audit Trail (incorporating AI Decision Logging)
- **Phase 20:** Performance Optimization & Caching (Redis)
- **Phase 21:** Deployment, Monitoring & Documentation

34. Risks & Mitigation

AI downtime mitigated by falling back to standard classification defaults and keyword indexing.
Connection leaks blocked using pooling and limit parameters.

35. Deliverables

Complete project structure template, environment configuration parameters, and detailed setup guides.

36. Acceptance Criteria

Clean monorepo configuration compiling without errors, repository paths verified, and configuration variables mapped.

37. Preparation for Phase 1

Align folder strategies and verify development database connectivity. The next phase will compile the baseline package integrations and routes.

© 2026 AI Helpdesk System Project. Confidential.